Status	Finished
Started	Friday, 21 August 2026, 11:24 AM
Completed	Friday, 21 August 2026, 11:28 AM
Duration	4 mins 19 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Write a program that given an integer ‘n’, prints the number of integers that are less than or equal to ‘n’ and co-prime to ‘n’

Two integers a and b are said to be relatively prime or co-prime if the only positive integer that evenly divides both of them is 1. That is, the only common positive factor of the two numbers is 1. This is equivalent to their greatest common divisor being 1.

Input Format:

One line containing the value of 'n', where 1<=n<=10,000

Output Format:

One line containing the number of integers that are co-prime to n and less than or equal to 'n'

Sample Test Cases

Test Case 1

Input

10

Output

4

Test Case 2

Input

23

Output

22

Test Case 3

Input

11

Output

10

Answer: (penalty regime: 0 %)

```
import math

def count_coprimes(n):
    count = 0
    # Iterate through all integers from 1 up to n
    for i in range(1, n + 1):
        # Two numbers are co-prime if their greatest common divisor (GCD) is 1
        if math.gcd(i, n) == 1:
            count += 1

    return count

# Reading input
n = int(input().strip())
print(count_coprimes(n))
```

	Input	Expected	Got	
✓	10	4	4	✓
✓	23	22	22	✓
✓	11	10	10	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Write a program to find the count of the number of prime numbers in a specified range.

The starting and ending number of the range will be provided as input to the program.

Assumption: 2 <=starting number of the range<= ending number of the range<=7919

Example1: If the starting and ending number or the range is given as 2 and 20, the program must return 8, because there are 8 prime numbers in the specified range from 2 to 20. namely (2. 3. 5, 7, 11, 13, 17, 19)

Example2: If the starting and ending number of the range is given as 700 and 725, the program must return 3, because there are 3 prime numbers in the specified range from 700 to 725, namely (701, 709, 719)

For example:

Input	Result
2 20	8
700 725	3

Answer: (penalty regime: 0 %)

```
def count_primes_in_range(start, end):
    # Helper function to check if a number is prime
    def is_prime(num):
        if num < 2:
            return False
        for i in range(2, int(num**0.5) + 1):
            if num % i == 0:
                return False
        return True

    # Count all prime numbers between start and end (inclusive)
    prime_count = 0
    for current in range(start, end + 1):
        if is_prime(current):
            prime_count += 1

    return prime_count

# Reading the starting and ending numbers from input
start_num = int(input().strip())
end_num = int(input().strip())

print(count_primes_in_range(start_num, end_num))
```

	Input	Expected	Got	
✓	2 20	8	8	✓
✓	700 725	3	3	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Given a number N, find the next perfect square greater than N.

Input Format:

Integer input from stdin.

Output Format:

Perfect square greater than N.

Example Input:

10

Output:

16

For example:

Input	Result
10	16
20	25

Answer: (penalty regime: 0 %)

```
import math

def find_next_perfect_square(n):
    # Find the integer square root of n
    root = int(math.isqrt(n))

    # The next perfect square will be the square of (root + 1)
    next_root = root + 1
    return next_root * next_root

# Reading input
num = int(input().strip())
print(find_next_perfect_square(num))
```

	Input	Expected	Got	
✓	10	16	16	✓
✓	20	25	25	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Given a positive integer a, find the smallest positive integer b whose multiplication of each digit equals to a.

If there is no answer or the answer is not fit in 32-bit signed integer, then return 0.

Example 1

Input:

48

Output:

68

Example 2

Input:

15

Output:

35

For example:

Input	Result
48	68
15	35

Answer: (penalty regime: 0 %)

```
def smallestFactorization(a):
    # If 'a' is less than 2, the smallest integer is 'a' itself
    if a < 2:
        return a

    res = []
    # Greedily divide by the largest possible single-digit factors (from 9 down to 2)
    for i in range(9, 1, -1):
        while a % i == 0:
            res.append(str(i))
            a //= i

    # If 'a' cannot be completely factored using single digits (prime factor > 9 exis
    if a > 1:
        return 0

    # Sort the digits in ascending order to make the resulting number as small as pos
    res.sort()
    ans = int("".join(res))

    # Check if the result fits within the 32-bit signed integer limit
    INT_MAX = 2**31 - 1
    if ans > INT_MAX:
        return 0

    return ans
```

```
27
28  # Reading input
29  a = int(input().strip())
30  print(smallestFactorization(a))
```

	Input	Expected	Got	
✓	48	68	68	✓
✓	15	35	35	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Write a program to find the count of non-repeated digits in a given number N. The number will be passed to the program as an input of type int.

Assumption: The input number will be a positive integer number >= 1 and <= 25000.

Some examples are as below.

If the given number is 292, the program should return 1 because there is only 1 non-repeated digit '9' in this number

If the given number is 1015, the program should return 2 because there are 2 non-repeated digits in this number, '0', and '5'.

If the given number is 108, the program should return 3 because there are 3 non-repeated digits in this number, '1', '0', and '8'.

If the given number is 22, the function should return 0 because there are NO non-repeated digits in this number.

For example:

Input	Result
292	1
1015	2
108	3
22	0

Answer: (penalty regime: 0 %)

```
from collections import Counter

def count_non_repeated_digits(n):
    # Convert the integer to a string to easily process individual digits
    digit_str = str(n)

    # Count the occurrences of each digit
    digit_counts = Counter(digit_str)

    # Count how many digits appear exactly once
    non_repeated_count = sum(1 for digit, count in digit_counts.items() if count == 1

    return non_repeated_count

# Reading input
num = int(input().strip())
print(count_non_repeated_digits(num))
```

	Input	Expected	Got	
✓	292	1	1	✓
✓	1015	2	2	✓
✓	108	3	3	✓
✓	22	0	0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

A Number is said to be Disarium number when the sum of its digit raised to the power of their respective positions becomes equal to the number itself. Write a program to print number is Disarium or not.

Input Format:

Single Integer Input from stdin.

Output Format:

Yes or No.

Example Input:

175

Output:

Yes

Explanation

1^1 + 7^2 +5^3 = 175

Example Input:

123

Output:

No

For example:

Input	Result
175	Yes
123	No

Answer: (penalty regime: 0 %)

```
def check_disarium(n):
    # Convert the number to a string to easily access each digit and its 1-based posi
    digit_str = str(n)

    # Calculate the sum of each digit raised to the power of its position (1-indexed)
    total_sum = sum(int(digit) ** (i + 1) for i, digit in enumerate(digit_str))

    # Check if the calculated sum equals the original number
    if total_sum == n:
        return "Yes"
    else:
        return "No"

# Reading input
num = int(input().strip())
print(check_disarium(num))
```

	Input	Expected	Got	
✓	175	Yes	Yes	✓
✓	123	No	No	✓
✓	89	Yes	Yes	✓
✓	90	No	No	✓
✓	518	Yes	Yes	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

An automorphic number is a number whose square ends with the number itself.

For example, 5 is an automorphic number because 5*5 =25. The last digit is 5 which same as the given number.

If it is an automorphic number display “Automorphic” else display “Not Automorphic”.

Input Format:

Take a Integer from Keyboard

Output Format:

Print Automorphic if given number is Automorphic number, otherwise Not Automorphic

Example input:

5

Output:

Automorphic

Example input:

25

Output:

Automorphic

Example input:

7

Output:

Not Automorphic

Answer: (penalty regime: 0 %)

```
def is_automorphic(n):
    # Calculate the square of the number
    square = n * n

    # Check if the string representation of the square ends with the number
    if str(square).endswith(str(n)):
        return "Automorphic"
    else:
        return "Not Automorphic"

# Reading input from the user
num = int(input().strip())
print(is_automorphic(num))
```

	Input	Expected	Got	
✓	5	Automorphic	Automorphic	✓
✓	625	Automorphic	Automorphic	✓
✓	7	Not Automorphic	Not Automorphic	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Let's print a chessboard!

Write a program that takes input:

Integer N(represents the rows and columns of a chessboard) and also the starting character of the chessboard

Output Format

Print the chessboard as per the given examples

Sample Input / Output

Input:

2

W

Output:

WB

BW

Answer: (penalty regime: 0 %)

```
def print_chessboard(n, start_char):
    # Determine the alternating character
    other_char = 'B' if start_char.upper() == 'W' else 'W'
    start_char = start_char.upper()

    for i in range(n):
        row_chars = []
        for j in range(n):
            # Alternates the character based on the sum of row and column indices
            if (i + j) % 2 == 0:
                row_chars.append(start_char)
            else:
                row_chars.append(other_char)
        print("".join(row_chars))

# Reading inputs
n = int(input().strip())
start_char = input().strip()

print_chessboard(n, start_char)
```

	Input	Expected	Got	
✓	2 W	WB BW	WB BW	✓
✓	3 B	BWB WBW BWB	BWB WBW BWB	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

You are choreographing a circus show with various animals. For one act, you are given two kangaroos on a number line ready to jump in the positive direction.

•The first kangaroo starts at position x1 and moves at a speed v1 meters per jump.

•The second kangaroo starts at position x2 and moves at a speed of v2 meters per jump and x2 > x1

•You have to figure out to get both kangaroos at the same position at the same time as part of the show before k jumps. If it is possible, return YES, otherwise return NO.

Input Format:

x1-position of kangaroo1

v1-Speed of kangaroo1

x2-position of kangaroo2

v2-Speed of kangaroo2

k-jumps

Output Format:

Both kangaroos are at the same position within k jumps, YES, otherwise NO.

For example:

Input	Result
0 3 4 2 6	YES

Answer: (penalty regime: 0 %)

```
def kangaroo(x1, v1, x2, v2, k):
    # If kangaroo 1 is behind kangaroo 2 (x2 > x1) and its speed is less than or equa
    # it will never be able to catch up.
    if v1 <= v2:
        return "NO"

    # We need to find if there is an integer number of jumps 'i' (where 1 <= i <= k)
    # x1 + i * v1 = x2 + i * v2
    # Rearranging: i * (v1 - v2) = x2 - x1 => i = (x2 - x1) / (v1 - v2)

    diff_x = x2 - x1
    diff_v = v1 - v2

    # Check if the distance can be evenly divided by the relative speed
    # and if the required jumps are less than or equal to k
    if diff_x % diff_v == 0:
        jumps = diff_x // diff_v
        if jumps <= k:
            return "YES"

    return "NO"

# Reading inputs
x1 = int(input().strip())
v1 = int(input().strip())
x2 = int(input().strip())
v2 = int(input().strip())
k = int(input().strip())

print(kangaroo(x1, v1, x2, v2, k))
```

	Input	Expected	Got	
✓	0 3 4 2 6	YES	YES	✓
✓	0 3 2 4 8	NO	NO	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Write python program to print the following pattern based on input size.

Input:

3

Output:

```
  1
 2 3
4 5 6
```

For example:

Input	Result
4	1 2 3 4 5 6 7 8 9 10

Answer: (penalty regime: 0 %)

```
def print_number_triangle(n):
    current = 1
    for i in range(1, n + 1):
        # Calculate leading spaces to center-align the triangle rows
        spaces = " " * (n - i)

        # Gather 'i' consecutive numbers for the current row
        row_elements = []
        for _ in range(i):
            row_elements.append(str(current))
            current += 1

        # Print the row with numbers separated by spaces
        print(spaces + " ".join(row_elements))

# Reading input
n = int(input().strip())
print_number_triangle(n)
```

	Input	Expected	Got	
✓	3	1 2 3 4 5 6	1 2 3 4 5 6	✓
✓	4	1 2 3 4 5 6 7 8 9 10	1 2 3 4 5 6 7 8 9 10	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.